

ASSIGNMENT 3.4 · FINAL DELIVERY

CareerHubX

Full-Stack Integration & Error Handling

"The product is the backend and the frontend together." — closing the gap between "works when everything goes right" and "works when anything goes wrong."

Typed API errors

Route error boundaries

422 → form mapping

Generated types

Sentry + CI

A walkthrough of what was built, why, and what remains for deployment · 2 July 2026

What this document is

A guided walkthrough of Assignment 3.4 as implemented in **this** repository. It records the written decisions, every change made to the code, how each was verified, and a precise checklist of the remaining steps that require your own accounts (Vercel, Sentry, GitHub, a deployed API).

DONE implemented & verified in the repo **YOU** needs your infrastructure/accounts

Repo-layout adaptation. The brief assumes a `client/` sub-folder and routes `/register`, `/dashboard`, `/dashboard/listings/[id]/applicants`. This project keeps the app at the **repo root** with real routes `/signup`, `/recruiter`, `/recruiter/jobs/[jobId]`. All paths here are the real ones.

Status at a glance

Part	Item	Status
2	Typed error foundation, client wiring, error boundaries, 422 mapping, Test 12	DONE
3	Generated types, DTO re-exports, 10 tsc breaks fixed, findings documented	DONE
5	<code>instrumentation.ts</code> (Sentry), CI workflow, <code>.env.example</code>	DONE (scaffold)
4	End-to-end demo on the deployed app + 12 screenshots	YOU
5	Deploy (Vercel + API), set <code>SENTRY_DSN</code> , green CI, CORS	YOU

Verified locally: `npx tsc --noEmit` → 0 errors · `npm run test:run` → 9 passed (8 prior + new Test 12), runner exits cleanly (no watch, no hang).

Test-suite hardening. `apply-gate.test.tsx` renders the apply page, which code-splits the wizard via `next/dynamic({ ssr:false })`. Under full-suite load the on-demand transform of that heavy module could outrun even the 15s `findBy` window, so the test intermittently saw only the loading skeleton. Fixed deterministically by eagerly importing the wizard module in the test — it lands in the transform/module cache before any test runs, so the page's dynamic import resolves from cache on the next tick. No production code touched.

Part 1 — Written decisions

Full answers live in `README.md`. The headlines:

Q1 · Most damaging error state

The **expired-session 401** during `/apply/[jobId]`. Five steps of effort are on the line; a blank screen there reads as data loss and causes abandonment. It is handled with a "Session Expired → Sign in" boundary — never a dead-end retry.

Q2 · Business rules — server vs client, and the duplicate

Rules 1–3 are enforced **both** sides; rules 4–5 are server-authoritative. Hiding the Apply button is **not** enough — the API also rejects a duplicate with **409**, surfaced as a toast carrying the API's own message (a whole-submission conflict, not a single bad field).

Q3 · Highest type-drift risk

`JobListingResponse` — its numeric fields and the `type` enum. Confirmed by the generator (Part 3).

Part 2 — API error handling

Step 1 · The typed error foundation — DONE

`src/lib/api-error.ts` defines `ApiError` extends `Error` with `status`, `code`, `fields`, the getters `isUnauthorized/isForbidden/isValidation`, and `parseApiError(res)` which reads the RFC 7807 body (wrapped in try/catch — some errors have no body) and maps status → code.

```
const code =
  res.status === 422 || (res.status === 400 && hasFieldErrors)
    ? "VALIDATION" // lift problem.errors → fields
    : statusToCode(res.status); // 401→UNAUTHORIZED, 403→FORBIDDEN,
                                // 404→NOT_FOUND, 409→CONFLICT, 5xx→SERVER_ERROR
```

Deviation, documented: the table maps only 422→VALIDATION, but ASP.NET model validation returns 400 with an `errors` map. So a 400 carrying field errors is also treated as VALIDATION — otherwise those messages would be lost.

Step 2 · API client uses parseApiError — DONE

Every authenticated function now does `throw await parseApiError(res)` instead of `throw new Error( ... )`. Public reads (`fetchJobs`, `fetchJob`) keep the generic `Error` — they surface at the route error boundary, not a form.

File	Functions converted
<code>src/lib/applicant-api.ts</code>	<code>applyToJob</code> , <code>fetchMyApplications</code> , <code>applicantLogin/Register</code>
<code>src/lib/employer-api.ts</code>	<code>createJobListing</code> , <code>updateApplicationStatus</code> , <code>fetchJobApplications</code> , <code>fetchCompanies/CompanyJobs</code> , <code>openApplicantCv</code> , <code>employerLogin/Register</code>

Proof: a duplicate application now shows the API's specific 409 message, not "Request failed: 409 Conflict".

Step 3 · Global error boundary — DONE

`src/app/error.tsx` — "use client", props `{error, reset}`, renders a heading + `error.message` + "Try again" (`reset()`) + "Go home". `useEffect(() => console.error(error))` is the Sentry hook.

Step 4 · Route-specific boundaries — DONE

Four route boundaries, each declaring its persona and delegating to a shared `RouteErrorScreen` that branches on `error.code`:

File	Persona / 403 heading
<code>src/app/apply/[jobId]/error.tsx</code>	Job Seeker Access Required
<code>src/app/applications/error.tsx</code>	Job Seeker Access Required
<code>src/app/recruiter/error.tsx</code>	Employer Access Required (= /dashboard)
<code>src/app/recruiter/jobs/[jobId]/error.tsx</code>	Employer Access Required (applicants)

The rule, enforced: UNAUTHORIZED and FORBIDDEN never render a retry button — re-rendering reproduces the identical error. They offer *Sign in / go back* instead. NOT_FOUND links back to the list; only the fallback (validation/server/unknown) shows "Try again".

Step 5 · 422 field errors → the wizard — DONE

`JobApplicationWizard`'s submit catch now detects `err instanceof ApiError && err.isValidation`, iterates `err.fields`, calls `form.setError()`, and jumps to the step owning the first invalid field via `STEP_FIELDS` — then a toast: "Please review your application — some fields need attention." A server field name is mapped to the wizard field the user edits (`coverNote/coverLetter` → `motivationText`).

Test 12 (`src/test/ApplicationWizard.test.tsx`) drives the whole wizard, mocks a 422 on submit, and asserts: it lands on the *Job-specific questions* step, shows the server message inline, and shows the review toast. **Passing.**

Steps 1–3 · Generate — DONE

```
# package.json
"generate:types": "openapi-typescript http://localhost:5080/openapi/v1.json -o src/types/api.generated.ts"
# run it
npm run generate:types → src/types/api.generated.ts (committed)
```

Before final submission: swap the localhost URL for your **deployed** API URL so the types reflect what production actually returns, then re-run and commit the diff.

Steps 4–5 · Re-export + fix breaks — DONE

`src/types/index.ts` now re-exports the two DTO mirrors from the generated file; the swap surfaced **10 tsc breaks**, all fixed by coercing at the wire→view boundary (no `as` suppression):

Field	Assumed	Generated	Fix
<code>type</code>	<code>EmploymentType</code>	<code>string</code>	<code>toEmploymentType()</code>
<code>salaryMin/Max</code>	<code>number null</code>	<code>number string null</code>	<code>Number(x)</code> + null-guard
<code>minimumExperienceYears</code>	<code>number</code>	<code>number string</code>	<code>Number(x)</code>
<code>applicantCount</code>	<code>number</code>	<code>number string</code>	<code>Number(x)</code>

What it revealed: the hand-written types were *stricter than the contract*. The API's OpenAPI types numbers as `number|string` and the job `type` as a bare `string`. Coercing once at the boundary keeps the app's strict view-model honest.

Part 5 — Delivery scaffolding (built for you)

Sentry — DONE (scaffold)

`src/instrumentation.ts` calls `Sentry.init()` for BOTH the `nodejs` and `edge` runtimes, DSN from `process.env.SENTRY_DSN`, `enabled` only when a DSN is present (a clean no-op otherwise), and exports `onRequestError`. `@sentry/nextjs` is a dependency.

CI — DONE (scaffold)

`.github/workflows/ci.yml` runs on push/PR: `npm ci` → `npx tsc --noEmit` → `npm run test:run`, in that order.

`.env.example` — DONE

Documents every variable (redacted): `NEXT_PUBLIC_API_BASE_URL`, `AUTH_SECRET`, `NEXT_PUBLIC_SITE_URL`, `SENTRY_DSN`, `NEXT_PUBLIC_SENTRY_DSN`.

What you must finish (needs your accounts)

1 · Deploy — YOU

- Deploy the **API** to a public HTTPS URL (Railway/Render/Azure).
- Deploy the **frontend** to Vercel.
- On Vercel set `NEXT_PUBLIC_API_BASE_URL` to the deployed API (not localhost) and `AUTH_SECRET` (fresh `npx auth secret`).
- Configure the API's **CORS** to allow your exact Vercel origin (verify in DevTools → Network → `Access-Control-Allow-Origin`).

2 · Sentry — YOU

- Create a Sentry project; copy its DSN.
- Set `SENTRY_DSN` (and `NEXT_PUBLIC_SENTRY_DSN`) as env vars on Vercel.
- Trigger a test error on the deployed app; confirm it appears in the Sentry dashboard within 60s.

3 · CI green — YOU

- Push to GitHub; open the Actions tab; confirm the **CI** workflow is green on `main`.

4 · Type-gen against production — YOU

- Point `generate:types` at the deployed `/openapi/v1.json`; re-run; commit the diff.

5 · End-to-end demo + 12 screenshots — YOU

Run both journeys (Employer + JobSeeker) on the deployed app in two sessions; screenshot each of the 12 milestones and paste them into the README. Milestones and pass criteria are listed in the README "End-to-end demo" section.

Command reference

```
npm run generate:types # regenerate api.generated.ts (swap URL for prod)
npx tsc --noEmit       # 0 errors
npm run test:run       # 9 passing, exits (no watch)
npm run build          # production build
npx auth secret        # generate AUTH_SECRET
```